

Week 1 Day 3: 로컬 정적 서버, 로그/설정, 재현성, RCA, AI 검증, spine 매핑

Overview

Day 3는 Day 2에서 준비한 저장소와 CLI 확인 기록을 사용해 로컬 정적 서버를 실행하고 관찰하는 날이다. 학생은 `python3 -m http.server`로 정적 파일을 제공하고, 브라우저와 `curl`로 HTTP 상태 코드를 확인하며, log, 설정, 비밀값, 실행 조건, 재현성, RCA, AI 검증 기준을 문서화한다.

중요한 제한: Day 3의 정규 확인 기록 학습은 로컬 정적 서버, 실행 조건, 재현성, RCA, AI 검증을 우선한다. 단, 8교시에는 Day 5 발표로 이어지는 별도 챌린지로 AI를 사용해 아주 작은 정적 웹사이트 초안을 만든다. 이 챌린지는 backend, database, paid API, authentication, cloud 배포 없이 HTML/CSS/JS 파일만 다룬다.

Learning Goals

- 로컬 정적 서버의 source, 실행 환경, command, 포트, data, 외부 의존성을 설명한다.
- log, 설정, 비밀값을 구분하고 비밀값 값을 노출하지 않는다.
- README에 start/check/stop/expected result/troubleshooting을 작성한다.
- 실패를 reproduce, observe, hypothesize, fix, recheck, prevent 순서로 기록한다.
- AI Coding Tool 답변을 공식 문서, 실행 결과, 보안/비용 기준으로 검증한다.
- 프로세스, file, 포트, HTTP, log 확인 기록을 Docker/Kubernetes/AWS/Terraform spine에 매핑한다.

오늘 반드시 가져갈 것

필수 개념	왜 필수인가	놓치면 생기는 문제	확인 기록
로컬 정적 서버	파일이 HTTP 응답으로 제공되는 최소 실행 단위다.	앱 구현과 서버 실행을 섞어 문제 원인을 찾지 못한다.	python3 -m http.server, 브라우저 URL, curl -I 상태 코드
실행 조건	source, 실행 환경, command, 포트, data, 외부 의존성이 있어야 다른 사람이 재현한다.	내 컴퓨터에서는 되지만 새 환경에서 실행되지 않는다.	실행 조건 6개 표
로그/설정/비밀값 구분	운영 기록, 실행 방식, 보호 대상은 다르게 다뤄야 한다.	토큰 노출, 잘못된 설정, 원인 분석 실패가 생긴다.	로그 예시, 설정 key, 비밀값 값 제외 메모
RCA와 재확인	수정했다는 말보다 같은 명령으로 다시 확인한 기록이 중요하다.	같은 장애가 반복되고 원인 후보가 남지 않는다.	404 재현, 수정, 재확인, 예방 기록
AI 답변 검증	AI 결과는 후보이며 운영 책임은 사람에게 남는다.	비용, 보안, 범위 초과 제안을 그대로 실행한다.	공식 문서, 실행 결과, 비밀값/cost/API 위험 표

Lesson Index

- 1교시: 로컬 정적 서버 실행 - python3 -m http.server, 브라우저/curl 확인
- 2교시: Log, 설정, 비밀값 - stdout/stderr, error message, env var, 비밀값 비노출
- 3교시: 서비스 실행 조건 - source, 실행 환경, command, 포트, data, 외부 의존성
- 4교시: 재현 가능성 - README, 경로, expected result, clean 디렉터리
- 5교시: 실패 분석 라이프사이클 - reproduce, observe, hypothesize, fix, recheck, prevent
- 6교시: 관찰 가능성과 배포 미리보기 - logs/상태 코드 확인 기록, build, 산출물, 릴리스, 배포, 되돌리기
- 7교시: AI Coding Tool 사용 원칙 - 공식 문서 확인, 실행 검증, 비밀값/cost/API 위험
- 8교시: Day 5 발표 챌린지 - AI로 간단한 정적 웹사이트 초안 만들기

Official References

Topic	Reference	오늘 확인할 키워드
Python http.server	https://docs.python.org/3/library/http.server.html	디렉터리, 포트, security warning
Python command line	https://docs.python.org/3/using/cmdline.html	-m module execution
MDN HTTP	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview	request, response, 상태 코드
Twelve-Factor App 설정	https://12factor.net/config	설정, env vars, credentials
GitHub README	https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes	start/check/troubleshooting
Google SRE Book	https://sre.google/sre-book/monitoring-distributed-systems/	monitoring, symptoms, causes

Required 확인 기록

확인 기록	제출 기준
Service execution contract	source/실행 환경/command/포트/data/외부 의존성
HTTP check	브라우저 또는 curl -I 상태 코드
Log/설정/비밀값 note	log example, 설정 key, 비밀값 비노출
Reproducibility checklist	clean 디렉터리 기준 실행 절차
RCA record	404 또는 포트 conflict 등 하나의 실패 분석
AI verification note	공식 문서/실행/비밀값/cost/API 위험 검증
AI website challenge draft	Day5 발표용 정적 웹사이트 초안, 실행 확인 기록, AI 사용 기록
Computing spine mapping	Docker/Kubernetes/AWS/Terraform 연결 표

Day 3 실습 루프

Day 3의 핵심은 "실행했다"가 아니라 "정상 결과와 실패 결과를 같은 방식으로 확인했다"이다. 학생은 작은 정적 서버 하나를 대상으로 정상 확인, 수정 확인, 의도적 오류, 오류 로그 확인, RCA 기록을 한번의 루프로 경험한다. 이 루프는 Week 2 Docker, Week 3 MSA, Week 4 Kubernetes, Week 5 AWS/Terraform에서 그대로 반복된다.

단계	해야 할 일	남길 evidence
1. 정상 실행	python3 -m http.server 8000으로 현재 디렉터리를 제공한다.	실행 command, 실행 경로, 서버 터미널 유지 여부
2. 정상 확인	브라우저 또는 curl -I http://localhost:8000으로 200 계열 응답을 확인한다.	URL, 상태 코드, server log의 GET / 요청
3. 내용 수정	index.html의 제목이나 본문 한 줄을 바꾼다.	변경 전/후 내용, 저장한 파일명
4. 수정 결과 확인	새로고침 또는 curl http://localhost:8000으로 수정된 본문이 보이는지 확인한다.	수정된 문구가 포함된 출력 또는 화면 기록
5. 의도적 오류	없는 파일인 /no-such-file.html을 요청해 404를 만든다.	failing command, 404 상태 코드
6. 오류 로그 확인	서버 터미널에서 실패 요청 log를 찾는다.	/no-such-file.html log excerpt
7. RCA 기록	원인 후보를 file 경로/resource 문제로 좁히고 정상 URL로 재확인한다.	reproduce/observe/hypothesize/fix/recheck/prevent 표

이 루프에서 중요한 점은 오류를 일부러 만든다는 사실 자체가 아니라, 실패가 발생했을 때 어떤 증거를 보고 판단을 좁히는지다. 404는 서버가 죽었다는 뜻이 아니다. 서버가 요청을 받았고, 요청한 경로의 resource를 찾지 못했다는 뜻이다. 반대로 connection refused는 서버 프로세스가 없거나 포트가 다를 때 먼저 의심한다.

수정 확인 최소 예시

```
printf '<!doctype html>\n<title>Week 1 Lab</title>\n<h1>Week 1 Local Service - Updated</h1>\n' > index.html
curl http://localhost:8000
```

기대 결과에는 Week 1 Local Service - Updated가 포함되어야 한다. 이 문구가 보이지 않으면 파일을 저장하지 않았는지, 서버를 다른 디렉터리에서 실행했는지, 브라우저 cache를 보고 있는지 순서대로 확인한다.

의도적 오류와 로그 확인 최소 예시

```
curl -I http://localhost:8000/no-such-file.html
```

기대 결과는 404 상태 코드다. 서버 터미널에는 /no-such-file.html 요청이 남아야 한다. 학생 기록에는 전체 로그를 무작정 붙여넣지 말고 요청 경로, 상태 코드, 시간처럼 원인 분석에 필요한 부분만 발췌한다.

비용/보안 주의사항

- 오늘은 로컬 서버만 실행한다. 외부 클라우드 리소스, 유료 API, 배포 서비스 가입을 요구하지 않는다.
- http.server는 학습용 로컬 정적 서버다. production web server로 사용하지 않는다.
- 비밀값 값은 README와 화면 공유에 기록하지 않는다. 설정 key 이름과 관리 원칙만 기록한다.

수업 종료 전 체크리스트

- [] `python3 -m http.server 8000`을 실행했다.
- [] `curl -I http://localhost:8000` 또는 브라우저로 상태 코드를 확인했다.
- [] 서버 터미널 로그에서 request 확인 기록을 찾았다.
- [] `index.html` 본문을 한 줄 수정하고 수정 결과가 실제 응답에 반영됐는지 확인했다.
- [] `/no-such-file.html` 요청으로 의도적 404를 만들고 오류 로그를 기록했다.
- [] 실행 조건 6가지를 표로 작성했다.
- [] README 재현성 절차를 작성했다.
- [] 하나의 실패를 RCA로 기록했다.
- [] AI 답변 검증표를 작성했다.
- [] 8교시 챌린지 산출물 또는 보류 사유를 기록했다.
- [] spine mapping 표는 Day5 통합 시간에 다시 완성한다.

챌린저 복구 기준

- 서버가 실행되지 않으면 먼저 `python3 --version`, 현재 경로, 포트 충돌을 확인한다.
- 브라우저가 비어 보이면 `curl -I`, `curl`, 서버 로그를 나란히 비교한다.
- 명령이 맞는지 확신이 없으면 README의 start/check/stop 순서만 다시 따라 한다.
- AI가 만든 제안이 커지면 backend, database, paid API, authentication, cloud 배포를 제외하고 정적 파일만 남긴다.

다음 주차 매핑

Day 3의 로컬 정적 서버는 이후 Docker image, Kubernetes Deployment/Service, AWS hosting, Terraform-managed infrastructure로 이어지는 가장 작은 실행 단위다. 8교시 챌린지 산출물은 Day5 발표에서 "AI가 만든 초안, 사람이 검증한 실행 조건, 남은 위험"을 설명하는 재료로 사용한다.

Visual Support



이 이미지는 Day3의 파일 작성, 실행 명령, 프로세스, 포트, HTTP 상태, 로그, README, RCA 흐름을 하나의 확인 기록 흐름으로 연결한다.